Sistema de Gestão de Ações – Documentação Completa

1. Introdução

Este projeto foi desenvolvido com o objetivo de simular um sistema de gestão de investimentos, utilizando Java com Spring Boot.

A aplicação permite o cadastro e consulta de corretoras e ações financeiras, integrando-se a APIs externas para garantir que todos os dados utilizados sejam reais, validados e consistentes.

Diferente de um sistema CRUD tradicional, o usuário não insere manualmente todas as informações. Ele fornece apenas dados básicos, como o CNPJ de uma corretora ou o ticker de uma ação, e o sistema se encarrega de:

- consultar serviços externos
- validar os dados obtidos
- transformar as informações para o padrão interno
- e somente depois persistir no banco

Essa abordagem aproxima o sistema de aplicações reais utilizadas no mercado financeiro.

2. Objetivos do Projeto

O objetivo principal é desenvolver uma API REST para gerenciamento de corretoras e ações com integração externa.

De forma mais específica, o projeto busca:

- implementar arquitetura em camadas
- consumir APIs externas
- validar dados automaticamente
- evitar inconsistências e duplicidade
- aplicar padrões de projeto
- organizar o código de forma clara e escalável

3. Resumo Arquitetural

O projeto foi desenvolvido utilizando Java 17 com o framework Spring Boot, adotando a Arquitetura em Camadas (Layered Architecture).

As camadas implementadas foram:

- Controller (Resource): exposição dos endpoints da API
- Service: regras de negócio e controle do fluxo
- Repository: acesso a dados com Spring Data JPA
- Domain (Entity): modelagem das entidades do sistema
- DTO: transferência segura de dados

Além das camadas exigidas, foram adicionados:

- mappers: conversão entre DTO e entidade
- integrations: isolamento das APIs externas
- ports: contratos para integração

Esses componentes adicionais elevam o nível técnico do projeto, garantindo melhor organização e desacoplamento.

4. Estrutura do Projeto

O sistema foi organizado em pacotes com responsabilidades bem definidas:

- domains → entidades e DTOs
- services → regras de negócio
- repositories → acesso ao banco
- resources → controllers
- integrations → APIs externas
- ports → interfaces de integração
- mappers → conversão de dados

5. Funcionamento das Camadas

Controller (resources)

Responsável por receber requisições HTTP, mapear endpoints e encaminhar dados para o service,não possui lógica de negócio.

Service (regra de negócio)

É o núcleo do sistema. Nessa camada são realizadas:

- validações
- regras de negócio
- chamadas para APIs externas
- montagem das entidades

Exemplo:

No cadastro de uma corretora, o service:

- valida o CNPJ
- consulta a API de CNPJ
- consulta o CEP
- aplica validações
- monta a entidade
- salva no banco

Repository

Responsável pela persistência dos dados.

Utiliza Spring Data JPA para operações como salvar, buscar e deletar.

Domain (Entity + DTO)

- Entity: representa as tabelas do banco
- DTO: controla entrada e saída de dados

Mappers

Responsáveis por converter DTO em entidade e vice-versa.

Ports

Interfaces que definem contratos para integração com APIs externas.

Integrations

Implementam os ports e realizam chamadas reais para APIs.

Cada integração possui:

- client → requisição HTTP
- response → estrutura da resposta
- adapter → conversão de dados

6. APIs Utilizadas

O sistema utiliza quatro APIs externas:

ViaCEP

Utilizada para obter dados de endereço a partir do CEP.

- gratuita
- não exige autenticação
- rápida e amplamente utilizada

BrasilAPI

Utilizada para obter dados de empresas a partir do CNPJ.

Retorna:

- razão social
- nome fantasia
- situação cadastral
- CNAE

Brapi

Utilizada para consulta de ações brasileiras.

Retorna:

- nome da empresa
- cotação
- moeda
- horário

TwelveData

Utilizada para consulta de ações internacionais.

Retorna:

- preço
- moeda
- data/hora

Estratégia adotada

- ações brasileiras → Brapi
- ações internacionais → TwelveData

Limitações consideradas

- dependência de internet
- possibilidade de indisponibilidade
- limites de requisição

7. Padrões de Projeto Utilizados

Strategy Pattern

Utilizado para escolher dinamicamente qual API de cotação será usada.

- BrapiStrategy → ações brasileiras
- TwelveDataStrategy → ações internacionais

Evita uso de if/else e facilita manutenção.

Adapter Pattern

Utilizado para adaptar os dados das APIs externas.

- BrasilApiAdapter
- ViaCepAdapter

Protege o sistema contra mudanças nas APIs.

Outros padrões

- Repository
- DTO
- Injeção de dependência

8. Regras de Negócio

Validação de corretora

Utiliza o CNAE retornado pela BrasilAPI para validar se a empresa atua no mercado financeiro.

Se compatível → validadaNaCvm = true

Prevenção de dados fictícios

O sistema ignora dados manuais enviados pelo usuário e utiliza apenas informações vindas das APIs.

Controle de duplicidade

- CNPJ duplicado → não permitido
- ticker duplicado → não permitido

9. Tratamento de Erros

Implementado com @ControllerAdvice.

Cenários tratados:

- erro de validação → 422
- duplicidade → 400
- API fora do ar → 503
- limite de requisição → 429

10. Fluxo do Sistema

Cadastro de corretora

CNPJ → BrasilAPI → Adapter → ViaCEP → Adapter → validação → banco

Cadastro de ação

Ticker → identificação de mercado → Strategy → API → Adapter → banco

Fluxo geral

Requisição
→ Controller
→ Service
→ API externa
→ Adapter
→ Repository
→ Banco
→ Resposta

11. Banco de Dados

- H2 (desenvolvimento)
- PostgreSQL / MySQL (produção)

12. Entidades

Corretora

- id
- cnpj
- razaoSocial
- nomeFantasia
- cep
- logradouro
- bairro
- cidade
- uf
- situacaoCadastral
- validadaNaCvm

Acao

- id
- ticker
- nomeEmpresa
- mercado
- moeda
- cotacaoAtual
- dataHoraCotacao

13. Decisão Arquitetural Importante

Optou-se por não vincular diretamente ações a corretoras.

Justificativa:

- evita consultas complexas
- melhora performance
- evita problemas de serialização
- segue princípios de desacoplamento

14. Diferenciais do Projeto

Cache

Uso de @Cacheable para melhorar performance de consultas ao ViaCEP.

Logs estruturados

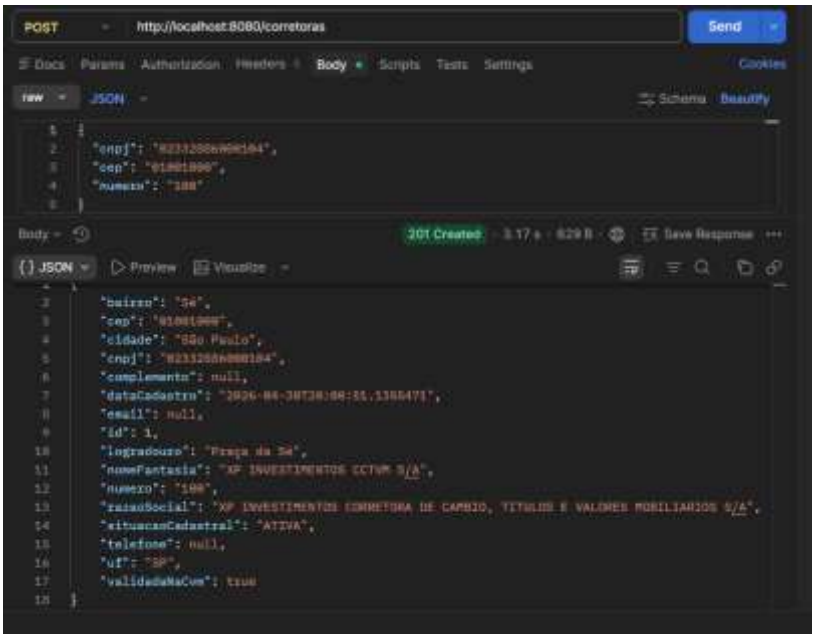
Uso de SLF4J para rastreamento de execução.

Documentação da API

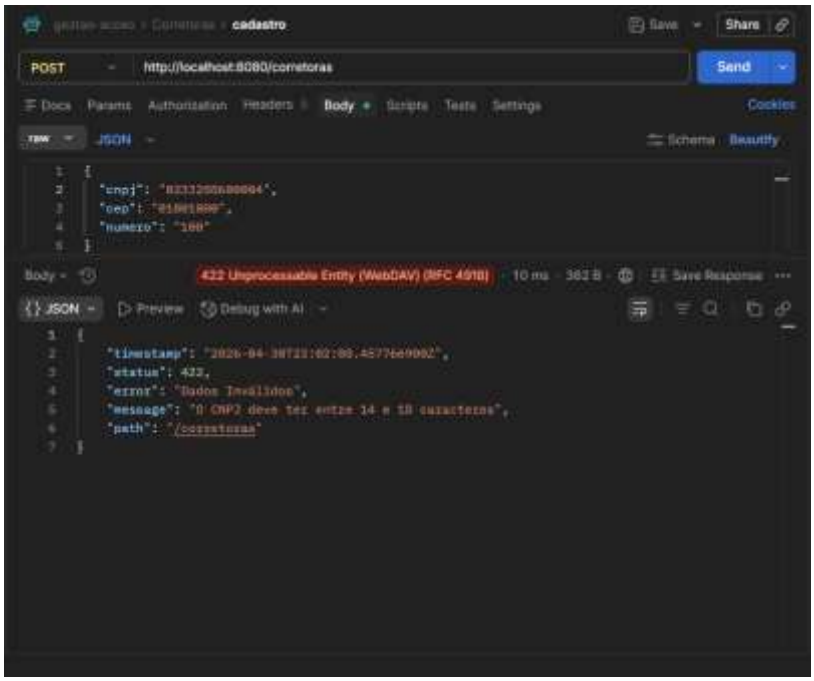
- Postman
- Swagger (OpenAPI 3.0), para esta função estar funcicionando a aplicação deve estar em execução no intrllij e acessar o navegador utilizando este link:http://localhost:8080/swagger-ui/index.html

Descrição dos testes de Endpoints no Postman

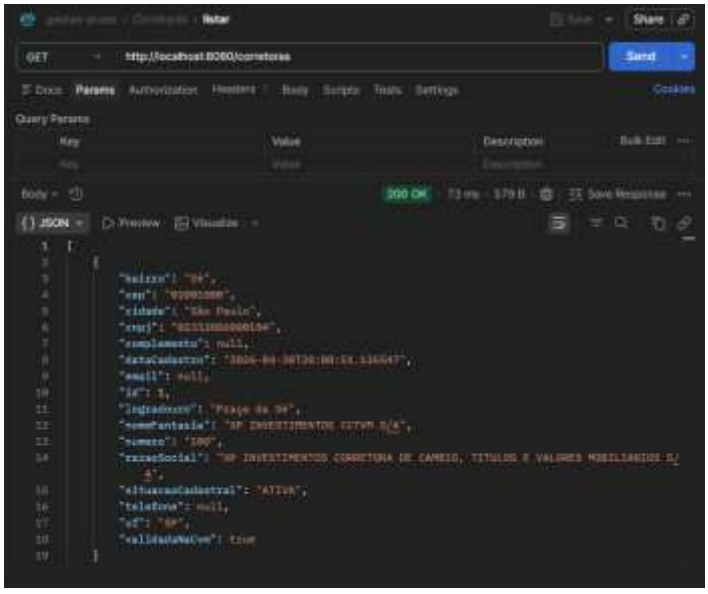
14.1 – Post das corretoras



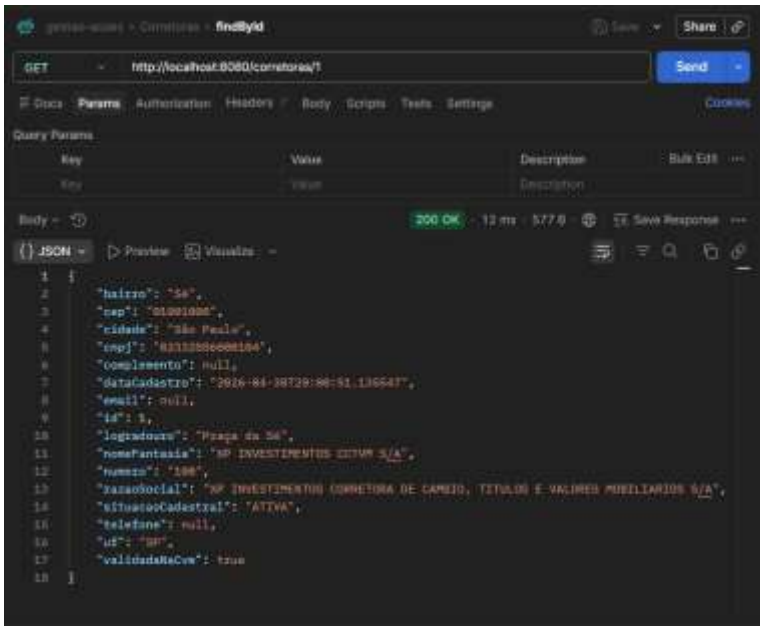
14.1.1 - Erro de dados inválidos ao inserir corretora no banco



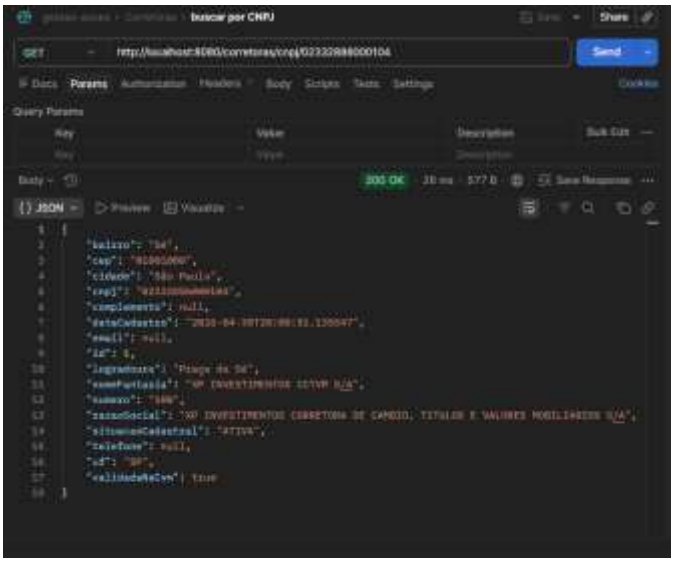
14.1.2 – FindALL das corretoras



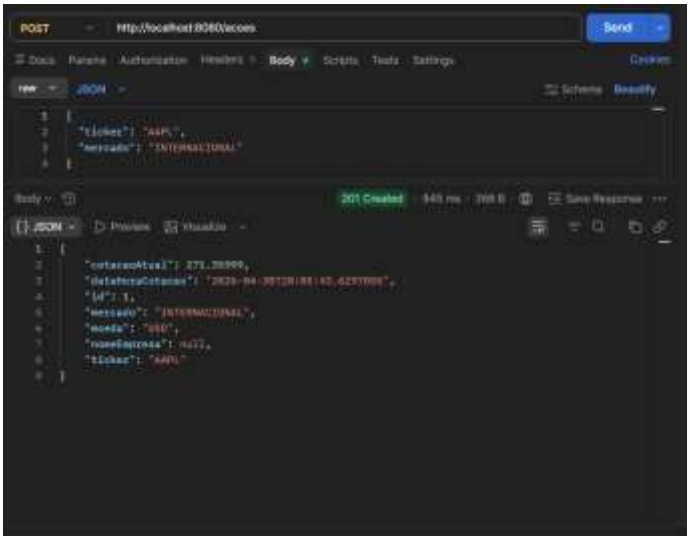
14.1.3 – FindById Corretora



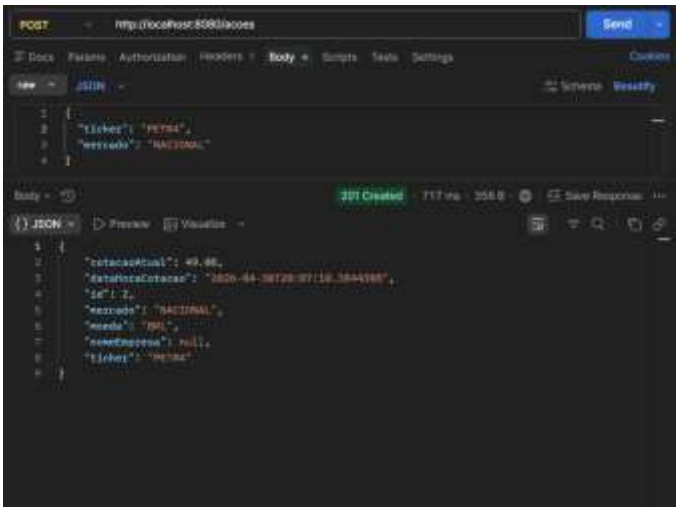
14.1.4 – Find por CNPJ



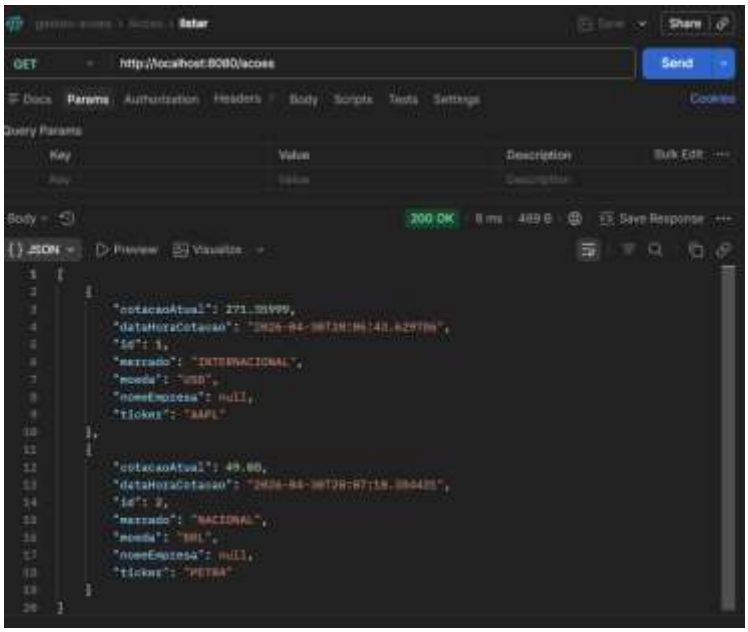
14.2 – Post ações internacionais



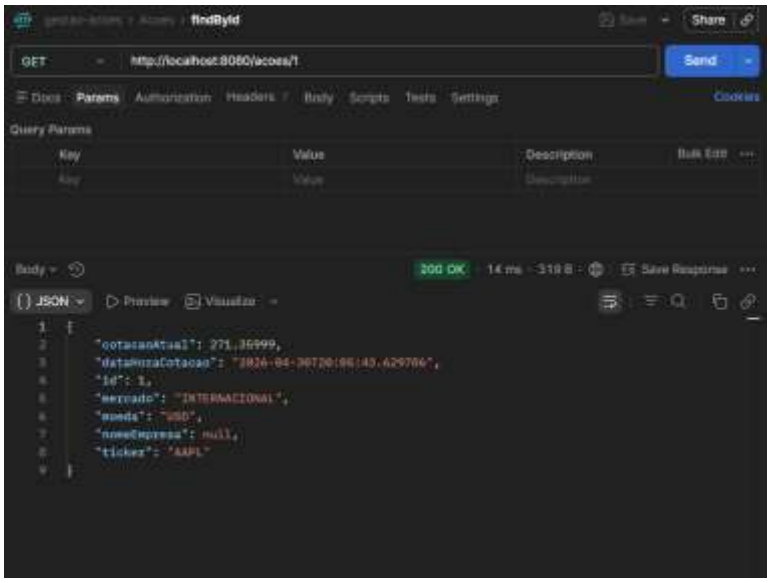
14.2.1 - Post ações Nacionais



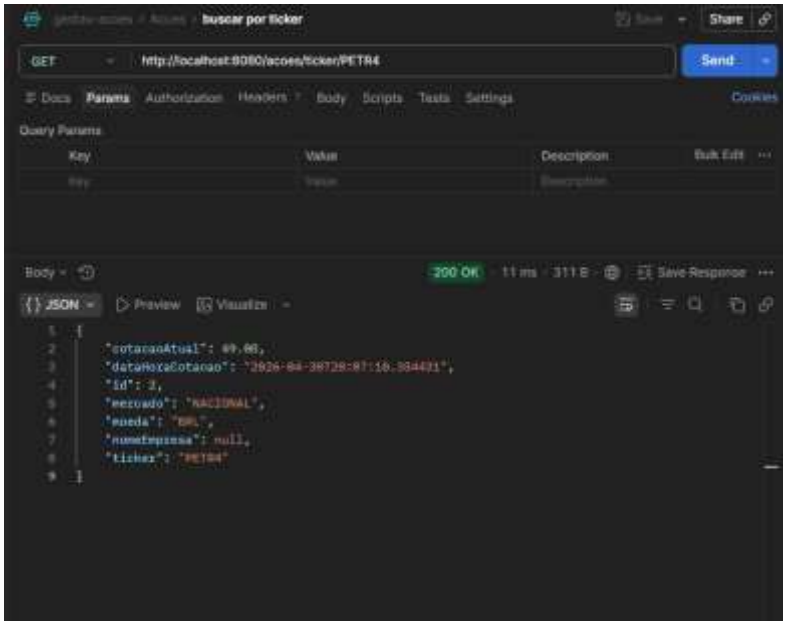
14.2.2 – Find ações



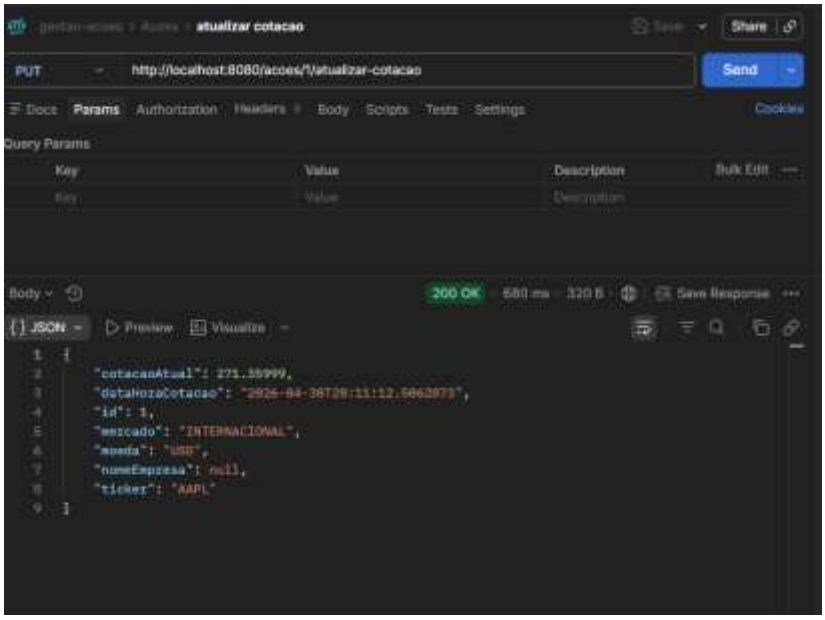
14.2.3 – FindByID por ID



14.2.4 – FindBy por Ticker



14.3 – Atualizar as cotações



15. Entregáveis

O projeto entrega todos os itens solicitados:

- código-fonte completo
- README documentado
- documentação das APIs
- coleção Postman
- testes endpoints No Postman
- sistema funcional para apresentação

16. Conclusão

O desenvolvimento deste projeto permitiu aplicar, na prática, diversos conceitos importantes de desenvolvimento backend e engenharia de software.

A utilização de uma arquitetura em camadas bem definida contribuiu para a organização do código e separação de responsabilidades, facilitando a manutenção e evolução do sistema.

A integração com APIs externas trouxe um nível maior de realismo para a aplicação, exigindo não apenas o consumo dos dados, mas também seu tratamento, validação e adaptação ao modelo interno.

Além disso, a aplicação de padrões de projeto como Strategy e Adapter foi fundamental para garantir desacoplamento e flexibilidade, permitindo que o sistema se adapte facilmente a mudanças futuras.

As regras de negócio implementadas, como validação de CNPJ, controle de duplicidade e validação de dados via APIs, garantem maior confiabilidade das informações armazenadas.

De forma geral, o projeto atende completamente aos requisitos propostos, demonstrando não apenas o funcionamento de uma API REST, mas também a aplicação de boas práticas, organização e integração com serviços externos.